

A Drift-Aware MLOps Framework for Multi-Model Serving: A Network Intrusion Detection Case Study

Hoang Tran-Nguyen-Viet¹, Thai Bui-Ngoc¹, Tuan Le-Anh¹,
Bao Nguyen-Van¹, and Quan Le-Trung¹

Faculty of Computer Networks and Communications,
University of Information Technology, VNU-HCM,
Ho Chi Minh City, Vietnam
{23520541,23521412}@gm.uit.edu.vn, {tuanla,baonv,quanlt}@uit.edu.vn

Abstract. Distribution shifts motivate a drift-aware MLOps framework for tenant-aware multi-model serving, using network intrusion detection (NIDS) as a case study rather than proposing a new classifier. Its PostgreSQL-backed Native Registry links model versions, references, and monitoring reports; asynchronous logs support drift monitoring, while update decisions are proposed for human review. We strengthen the evaluation with a fixed BENIGN/DDoS XGBoost model across disjoint pools, 120 windows, two sizes, and four alert thresholds. Changing attack prevalence produces strong distribution alerts while macro-F1 remains near one. Replacing one quarter of attack samples with held-out families reduces mean attack recall to 0.7533 for 1,000-sample windows, although a BH-adjusted KS monitor alerts in only one of five windows at a drift-share threshold of 0.30. A four-window service replay supplies separate integration evidence. These results support treating distribution alerts as lifecycle evidence for review rather than automatic update decisions; we claim no new detector, registry technology, or demonstrated adaptation policy.

Keywords: MLOps · Data Drift · Multi-model Serving · Network Intrusion Detection

1 Introduction

Production traffic can change while quality labels arrive late, challenging NIDS generalization [16]. We present a tenant-aware MLOps framework linking serving versions to logs, reference data, drift monitoring, and review, using NIDS as a case study.

We contribute (i) a tenant-aware control-plane/data-plane architecture with a PostgreSQL-backed Native Registry for model-version state; (ii) a drift-aware lifecycle formulation separating distribution evidence, labeled quality, and reviewed actions; and (iii) a controlled fixed-model NIDS evaluation and service replay. The evaluation asks *when constructed changes make distribution alerts*

agree or disagree with lower predictive quality; this concerns changed composition, not deterioration over time. The shift/harm distinction is established [?].

2 Related Work

MLOps research has examined lifecycle architectures, maturity, software engineering, technical debt, and production data management [1,4,3,2,9]. TFX, MLflow, Clipper, and pipeline platform studies supply pipeline, lifecycle, and serving foundations [6,12,13,5]; integrating these responsibilities is not itself our novelty.

Rabanser et al. distinguish shift detection from harmfulness [?]; our evaluation quantifies their disagreement for controlled NIDS mixtures. Sridhar et al. address production governance and prediction provenance [?]; we apply that concern to model/reference/window/report associations. Leest et al. organize contextual information for interpreting monitoring [?]; version and window identities supply a limited instance of such context.

HarmonE monitors accuracy, energy, and distribution shifts to guide adaptation [18]. We evaluate disagreement before an update decision. Concept-drift analyses [10,11] and continual-learning NIDS [19] address changing concepts or updates; our classifier stays fixed. Our addition is a framework-level instantiation with controlled evaluation, not a new distinction between shift and harm.

3 Drift-Aware MLOps Framework

Figure 1 and Table 1 summarize the workflow and implementation status.

3.1 Control Plane and Native Registry

The Django/DRF control plane stores lifecycle metadata in PostgreSQL. Each tenant owns projects grouping model families; version UUIDs link artifacts, flavor, stage, and deployability. Deployment/endpoint records supply runtime addresses and health state. Metadata alone does not establish runtime readiness.

Native Registry denotes product-state ownership, not new registry technology. A monitor links a version to a reference asset; run UUIDs identify status, scores, and report URIs. Reference UUIDs have optional checksums; snapshot/window links are optional. They do not guarantee immutable reference bytes, frozen membership, or consistency under concurrent changes. Evidence-linked review remains a proposed design.

3.2 Data Plane and Multi-Model Serving

The FastAPI gateway resolves versions and checks private-model access through project API keys or bearer identity; public models permit public invocation. Workers load or reuse version-associated models and return predictions with optional confidence.

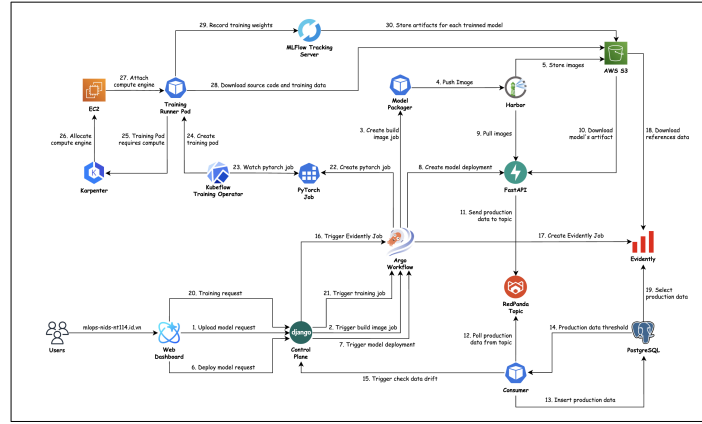


Fig. 1. Platform workflow: packaging, deployment, prediction logging, monitoring, and training. Training and compute paths show configured scope; their effectiveness is unevaluated.

Multi-model serving is architectural scope: multiple versions can be routed, but this study evaluates one classifier, not tenant performance isolation, cache efficiency, scalability, or concurrency robustness.

3.3 Production Logging and Evidence Collection

Successful inference schedules a background event for Redpanda; a consumer persists production records in PostgreSQL. A compact representation is

$$z_t = (u, i, x_t, \hat{y}_t, c_t, t),$$

where u, i identify tenant/version, $x_t, \hat{y}_t$ input/prediction, c_t optional confidence, and t the timestamp. Events also carry project, prediction, and request IDs; stored version/project relations preserve ownership context.

Persistence is outside the synchronous prediction response, whose success does not guarantee durable logging. Exactly-once delivery, retry idempotency, and complete failure recovery are not established; the replay audits final coverage.

3.4 Drift and Quality Evidence

For tenant u and version i , let $\hat{y} = f_{u,i}(x)$, with reference $D_{\text{ref}}^{u,i}$ and window $D_{\text{cur}}^{u,i}$. Reports should retain these identities, membership, configuration, and run context. The deployed reader selects the latest version-scoped records; membership can change. Offline manifests fix membership; a separate audit reconciles pilot inputs with stored records.

Table 1. Responsibility/status: implemented (I), defined (D), evaluated (E), or live replay only (R). I does not imply operational guarantees.

Responsibility	Owner	State/evidence	Status
Identity	Native Registry	Project, version, artifacts	I
Serving	Gateway, worker	Version, prediction	I
Logging	Broker, consumer	Version-associated records	I
Monitoring	Registry, monitor	Reference, run, report; optional window links	I, limited
Distribution	KS/BH; Evidently	Feature decisions, alert	E; R
Quality	Evaluation layer	Macro-F1, recall, FPR	E
Review/action	Review procedure	Evidence-linked decision	D
Update policy	Update workflow	Candidate, gates, action	D, not E

Offline tests yield p_j for d features. Applying Benjamini–Hochberg (BH) [?] within each window gives adjusted p-values q_j :

$$I_j = \mathbf{1}[q_j < 0.05], \quad \rho = \frac{1}{d} \sum_{j=1}^d I_j, \quad A = \mathbf{1}[\rho \geq \tau_D]. \quad (1)$$

The flagged-feature fraction ρ depends on the rule, feature set, and sample size; it is neither shift severity, affected-request share, nor failure probability. Marginal tests can miss dependence changes. BH here does not guarantee dataset-level false-alert control under dependent, tied features.

Live Evidently uses a separate `DataDriftPreset` configuration (Sect. 5.3), not replication of KS/BH. Quality evidence requires ground truth matched to predictions for macro-F1, attack recall, and benign false-positive rate (FPR). This study scores labels in the evaluation layer; automatic production labeling is not demonstrated. Missing labels leave quality unknown.

3.5 Review and Lifecycle Action Boundary

An alert requests inspection, not replacement. Low labeled quality warrants review even when $A = 0$. Actions should reference their supporting report and quality window. Retraining requires representative labels; promotion also requires held-out quality, operational gates, and human review. These are proposed conditions, not certified enforcement or evaluated policies.

3.6 Prototype Implementation and Interface Views

Object storage holds artifacts and reports; MLflow optionally supports tracking and loading. Docker or Argo backends execute build, deployment, training, and drift jobs; Argo/Kubeflow resources support cluster execution. These adapters do not own product state, and not all paths are evaluated. Figure 2 shows the prediction and drift-monitoring prototype interfaces.

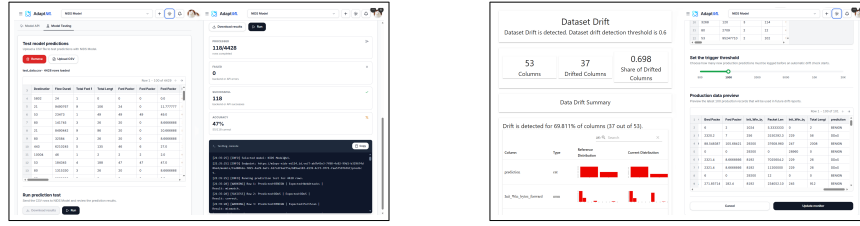


Fig. 2. Prototype interfaces: (a) CSV prediction testing; (b) Evidently drift monitoring. UI values are illustrative; the monitor’s prediction field is outside the 52-feature replay.

4 NIDS Case Study: Controlled Evaluation

4.1 Provenance and Disjoint Pools

The case study evaluates the framework’s monitoring assumptions on labeled CICIDS2017 network flows [15]. We use a team-subsampled derivative of *CICIDS2017: Cleaned & Preprocessed* dataset, followed by the team’s *MLOps NIDS Dataset*.¹ Local extraction caps labels, reserves test rows, and shuffles the remainder. Its `reference_data.csv` contains 209,865 rows: 100,000 BENIGN, 50,000 DDoS, 50,000 PortScan, 8,150 BruteForce, and 1,715 WebAttacks, with 52 numeric features.

Upstream preprocessing combines data, cleans values, groups labels, and selects features before our split. Time/capture identifiers are absent. Potential upstream leakage and complete byte-level lineage from the original release remain unverified in this study.

We hash ordered numeric feature vectors, excluding labels and normalizing signed zero. Twenty rows in ten feature-identical, conflicting-label groups and one further duplicate are removed; there are no nonfinite rows. The remaining 209,844 rows are stratified into train/development/reference/evaluation pools at 50/10/20/20% with split seed 260926. All six pool pairs have zero exact feature-vector overlap, including benign rows. This does not exclude session-level or correlated-flow dependence.

4.2 Fixed Model and Window Construction

One binary XGBoost classifier [14] is fitted on 74,994 BENIGN/DDoS rows from the train pool. PortScan, BruteForce, and WebAttacks are deliberately unseen during fitting. We use 200 trees, depth 6, learning rate 0.1, full row/column sampling, the histogram tree method, model seed 42, and prediction threshold 0.5. No scaler, SMOTE, or evaluation-based tuning is used; two development windows check execution. The model stays fixed.

¹ Kaggle source: <https://www.kaggle.com/datasets/ericanacletoribeiro/cicids2017-cleaned-and-preprocessed>; team derivative: <https://www.kaggle.com/datasets/hongtrnnguyenvit/nids-reference-data>.

Table 2. Controlled scenarios; S2 changes attack composition at fixed prevalence.

Scenario	Total attack fraction	Held-out share of attack rows
S0	30%	0% (all DDoS)
S1	15%, 50%, 85%	0% (all DDoS)
S2	30%	25%, 50%, 100%

A separate 5,000-row reference contains 70% BENIGN and 30% DDoS (reference sampling seed 260927). Evaluation windows contain $n \in \{500, 1000\}$ rows sampled without replacement within each window. Table 2 specifies the scenarios. In S2, replacement attacks are divided as evenly as possible among the three held-out families. Thus S2 at 25% replacement and $n = 1000$ contains 700 benign, 225 DDoS, and 75 held-out-family rows. Replacement percentage is a fraction of *attack rows*, not of the entire window.

Seeds are 42, 123, 456, 789, and 1024 per setting; S0 adds 2000–2024, giving 30 windows per size and 120 overall. Windows reuse the evaluation pool with fixed class counts. SD describes sampling with one model/reference, not independent captures or fits.

4.3 Detection, Metrics, and Reproducibility

Two-sided asymptotic Kolmogorov–Smirnov tests cover all 52 features (6,240 tests), followed by Eq. 1. We compare thresholds 0.10, 0.30, 0.50, and 0.70, retaining the original study threshold of 0.30 as the principal setting; no threshold is selected from the revised outcomes. Uncorrected tests serve as a comparison. Each window yields macro-F1, attack recall, benign FPR, ROC-AUC, family recall, and KS statistics.

The retained local run `run_20260926` includes `dataset_manifest.json` (input SHA-256, ordered schema, exclusions), `model_manifest.json` (model SHA-256), compressed split/window memberships, and `reference_manifest.csv`. Config/environment files preserve seeds, hyperparameters, and versions, including XGBoost 3.2.0, SciPy 1.17.1, and scikit-learn 1.8.0. Per-window metrics and feature tests feed the summary tables; `build_result_assets.py` formats the main table. The separate `verify_results.py` reloads the model, recomputes predictions and all KS tests, and checks BH against SciPy; its retained verification record reports all checks passed. This supports numerical checking with the frozen local input, not independent reconstruction from Kaggle. Public artifact access and versioned input acquisition are not established; upstream byte-level lineage remains unverified.

5 Results

5.1 Distribution Drift and Predictive Quality

Table 3 reports the five principal windows per scenario at $n = 1000$. S1 changes prevalence while retaining the learned attack family. Despite drift shares of

Table 3. Fixed-model results for 1,000-row windows: mean $\pm$ sample SD over five seeds. Alerts use $\tau_D = 0.30$; S0 alert count uses all 30 stable windows.

Scenario	Drift share	Macro-F1	Attack recall	Alerts
S0	0.000 $\pm$ 0.000	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000	0/30
S1 (15%)	0.742 $\pm$ 0.059	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000	5/5
S1 (50%)	0.850 $\pm$ 0.077	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000	5/5
S1 (85%)	0.981 $\pm$ 0.000	0.999 $\pm$ 0.001	1.000 $\pm$ 0.001	5/5
S2 (25%)	0.165 $\pm$ 0.202	0.905 $\pm$ 0.002	0.753 $\pm$ 0.004	1/5
S2 (50%)	0.865 $\pm$ 0.073	0.789 $\pm$ 0.003	0.508 $\pm$ 0.005	5/5
S2 (100%)	0.981 $\pm$ 0.000	0.422 $\pm$ 0.007	0.010 $\pm$ 0.007	5/5

0.7423–0.9808, macro-F1 remains at least 0.9992. These alerts correctly reflect changed input composition; unchanged predictive quality does not make them false distribution alerts.

S2 instead tests held-out attack families. At 25% replacement, attack recall falls to 0.7533, while the principal detector alerts in only 1/5 windows. The same 1/5 count occurs at $n = 500$, where mean recall is 0.7520. These held-out attacks occupy only 7.5% of the full window: substantial recall loss within the attack class can coexist with infrequent dataset alerts. At full replacement, recall is 0.0100 for $n = 1000$: PortScan and WebAttacks recall are zero and BruteForce recall averages 0.0300. This is a limitation of the deliberately restricted BENIGN/DDoS baseline, not an evaluation of a model trained on all families.

The S2 trend must be interpreted as a mixture effect: for replacement fraction r , aggregate attack recall equals $(1 - r)R_{\text{DDoS}} + rR_{\text{held-out}}$. Post-hoc inspection of the full evaluation pool finds 9,999/10,000 DDoS, 4/9,998 PortScan, 60/1,630 BruteForce, and 0/344 WebAttacks predicted as attacks. Increasing poorly recognized families therefore explains the decline; it does not demonstrate loss of DDoS knowledge over time. This inspection reuses the evaluation pool and is not an independent test.

S1 at 85% prevalence and S2 at full replacement both flag approximately 98% of features, yet their macro-F1 values differ greatly. Drift share thus cannot rank predictive harm in these cases. Across all 30 S0 windows at $n = 1000$, mean macro-F1 is 0.999881, recall 0.999889, and FPR 0.000095; perfect values in the five-window table do not hold for every stable draw. S1 at 85% prevalence has mean benign FPR 0.001333 in the principal windows; the other table rows have zero observed FPR.

5.2 Threshold and Window-Size Sensitivity

Table 4 shows alert counts, with 30 windows for each S0 size and five for each shifted setting. All 60 S0 windows have zero BH alerts across the tested thresholds; this finite, conditional observation is not a zero-false-alarm guarantee. At 25% replacement and $n = 1000$, lowering τ_D from 0.30 to 0.10 increases alerts

Table 4. BH-corrected alert counts across drift-share thresholds. S1 labels give total attack prevalence; S2 labels give the fraction of attack rows replaced by held-out families. S0 uses 30 windows per size; other scenarios use five.

Scenario	$n = 500$				$n = 1000$			
	.10	.30	.50	.70	.10	.30	.50	.70
S0	0/30	0/30	0/30	0/30	0/30	0/30	0/30	0/30
S1 (15%)	5/5	5/5	5/5	3/5	5/5	5/5	5/5	3/5
S1 (50%)	5/5	5/5	5/5	4/5	5/5	5/5	5/5	5/5
S1 (85%)	5/5	5/5	5/5	5/5	5/5	5/5	5/5	5/5
S2 (25%)	1/5	1/5	0/5	0/5	2/5	1/5	1/5	0/5
S2 (50%)	5/5	5/5	4/5	1/5	5/5	5/5	5/5	5/5
S2 (100%)	5/5	5/5	5/5	5/5	5/5	5/5	5/5	5/5

from 1/5 to 2/5, but does not eliminate missed shifts. At 50% replacement and $\tau_D = 0.70$, alerts increase from 1/5 for $n = 500$ to 5/5 for $n = 1000$.

Correction also changes sensitivity. Without BH, S0 at $n = 1000$ and $\tau_D = 0.10$ produces 3/30 alerts instead of 0/30. Conversely, S2 at 25% replacement and $\tau_D = 0.30$ produces 5/5 alerts instead of 1/5. This trade-off should be reported rather than choosing a test after observing its outcomes. Window size affects statistical power; these non-temporal windows do not measure detection delay.

5.3 Controlled Service Replay

Four seed-42, 1,000-row windows used identical model bytes and the 5,000-row reference under separate version UUIDs, each initially without production records. S1 used 85% DDoS. Labels were withheld until scoring. Each export had 1,000 unique prediction IDs and request IDs; exact 52-feature-vector matching established one record per frozen input. The browser did not send the handoff row ID, so reconciliation used vectors, not retry identity.

S0's consumer stored 700 class-0 predictions as empty strings. Scoring used all 1,000 UI predictions cross-checked against database records, allowing only that empty-string/zero discrepancy. S1/S2 used post-fix database predictions without blanks. All four prediction sets matched the offline artifact. S1 incurred 42 HTTP 502/503 failures during a worker interruption: recovery submitted those inputs plus 824 unsent inputs, yielding 1,000 records from 1,042 attempts. The audit establishes final coverage, not retry idempotency or unique failure causes.

The service source pins Evidently 0.4.15 and uses `DataDriftPreset` without feature-test overrides. With 5,000 reference rows, its defaults are normalized Wasserstein distance for numeric columns with more than five distinct values, Jensen–Shannon distance otherwise, and feature threshold 0.1, without BH. This source configuration is not an archived runtime dependency inventory. Reports used dataset threshold 0.60 with all 52 features present. Table 5 is separate integration evidence, not replication of offline KS/BH.

Table 5. Single-window live replay per setting; dataset threshold 0.60.

Window	Flagged features	Alert	Attack recall
S0	1/52	No	1.0000
S1 (85%)	43/52	Yes	1.0000
S2 (50%)	22/52	No	0.5067
S2 (100%)	29/52	No	0.0100

6 Discussion and Limitations

S1 alerts without material measured harm; offline S2-r25 remains unflagged in four of five windows at $\tau_D = 0.30$. The reported live S2-r50/r100 windows remain unflagged at service threshold 0.60 despite low attack recall. Hence the framework treats alerts as review evidence, not update commands.

Because labels may arrive after monitoring, a distribution report records input shifts without measuring missed attacks. Labeled recall later provides that separate quality evidence, as the contrasting S1 and S2 windows illustrate.

Upstream selection, missing time identifiers, correlated flows, and familiar-class separability limit generalization despite exact deduplication. Five shifted windows per setting, fixed class counts, one model/reference, and tied-feature tests limit inference. S2 changes composition, not demonstrated temporal $P(Y | X)$; natural drift and adaptation require separate evaluation.

The pilot verifies neither frozen associations under concurrency nor failure recovery. Its HTTP errors and the earlier 89-second Locust replay establish neither availability nor scalability. Multi-model load, lifecycle ablations, and promotion/rollback benefits remain unevaluated. Ownership checks do not sandbox uploaded code; isolation remains untested and requires restricted artifact formats, isolated identities, network/resource limits, and adversarial testing.

7 Conclusion

We presented a drift-aware MLOps framework for tenant-aware multi-model serving whose Native Registry associates model versions, references, and monitoring reports. A controlled NIDS evaluation and a service replay strengthen the prototype evidence: because alerts and labeled quality can disagree, the framework treats alerts as review evidence, not update decisions. Update effectiveness and operational guarantees remain open.

Acknowledgements This research was supported by the VNUHCM-University of Information Technology’s Scientific Research Fund.

References

1. R. Sommer and V. Paxson, “Outside the Closed World: On Using Machine Learning for Network Intrusion Detection,” in *Proc. IEEE Symposium on Security and Privacy*, 2010, pp. 305–316.
2. S. Rabanser, S. Günnemann, and Z. Lipton, “Failing Loudly: An Empirical Study of Methods for Detecting Dataset Shift,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019.
3. M. Kreuzberger, N. Kuhl, and S. Hirschl, “Machine Learning Operations (MLOps): Overview, Definition, and Architecture,” *IEEE Access*, 11, 31866–31879, 2023.
4. M. M. John, H. H. Olsson, and J. Bosch, “Towards MLOps: A Framework and Maturity Model,” in *Proc. SEAA*, 2021, pp. 334–341.
5. S. Amershi et al., “Software Engineering for Machine Learning: A Case Study,” in *Proc. ICSE-SEIP*, 2019, pp. 291–300.
6. D. Sculley et al., “Hidden Technical Debt in Machine Learning Systems,” in *NeurIPS*, vol. 28, 2015.
7. N. Polyzotis, S. Roy, S. E. Whang, and M. Zinkevich, “Data Management Challenges in Production Machine Learning,” in *Proc. ACM SIGMOD*, 2017, pp. 1723–1726.
8. D. Baylor et al., “TFX: A TensorFlow-Based Production-Scale Machine Learning Platform,” in *Proc. 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2017, pp. 1387–1395.
9. M. Zaharia et al., “Accelerating the Machine Learning Lifecycle with MLflow,” *IEEE Data Engineering Bulletin*, vol. 41, no. 4, pp. 39–45, 2018.
10. D. Crankshaw et al., “Clipper: A Low-Latency Online Prediction Serving System,” in *Proc. USENIX NSDI*, 2017, pp. 613–627.
11. Y. Zhou, Y. Yu, and B. Ding, “Towards MLOps: A Case Study of ML Pipeline Platform,” in *Proc. ICAICE*, 2020, pp. 494–500.
12. V. Sridhar, S. Subramanian, D. Arteaga, S. Sundararaman, D. Roselli, and N. Talagala, “Model Governance: Reducing the Anarchy of Production ML,” in *Proc. USENIX Annual Technical Conference*, 2018, pp. 351–358.
13. J. Leest, C. Raibulet, P. Lago, and I. Gerostathopoulos, “From Tea Leaves to System Maps: A Survey and Framework on Context-Aware Machine Learning Monitoring,” *IEEE Trans. Softw. Eng.*, vol. 51, no. 12, pp. 3218–3246, 2025.
14. H. Bhatt et al., “HarmonE: A Self-Adaptive Approach to Architecting Sustainable MLOps,” arXiv:2505.13693, 2025.
15. J. Gama et al., “A Survey on Concept Drift Adaptation,” *ACM Comput. Surv.*, vol. 46, no. 4, pp. 1–37, 2014.
16. G. I. Webb, R. Hyde, H. Cao, H.-L. Nguyen, and F. Petitjean, “Characterizing Concept Drift,” *Data Mining and Knowledge Discovery*, vol. 30, no. 4, pp. 964–994, 2016.
17. X. Zhang et al., “Continual Learning with Strategic Selection and Forgetting for Network Intrusion Detection,” arXiv:2412.16264, 2025.
18. Y. Benjamini and Y. Hochberg, “Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing,” *J. R. Stat. Soc. B*, vol. 57, no. 1, pp. 289–300, 1995.
19. I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization,” in *Proc. ICISSP*, 2018, pp. 108–116.
20. T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proc. ACM KDD*, 2016, pp. 785–794.